

EDS Lab Assignments

Practical 4

Name: Jay Vilas Mali

PRN: 202501040023

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

The user should enter a list of numbers separated by space when prompted.

Output Format:

The program should display the mean of even and odd numbers separately. Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

1

import pandas as pd

2

3

Take inputs from the user to create a list of numbers

numbers = list(map(int, input().split()))

4

5

6

Create a Pandas series from the list of numbers

series = pd.Series(numbers)

7

8

Grouping by even and odd numbers and calculating the mean

grouped = series.groupby(series%2==0).mean()

9

10

11

Display the mean of even and odd numbers with labels

grouped.index = ['Even' if is_even else 'Odd' for is_even in

12

grouped.index]

13

print("Mean of even and odd numbers:")

Average time

0.010 s

10.17 ms

Maximum time

0.021 s

21.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd --- 5.0

Even --- 6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd --- 5.0

Even --- 6.0

dtype: float64

Terminal

Test cases

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).

- Add the new row to the DataFrame.

- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.

- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.

- Remove the specified row.

- Display the DataFrame after deleting the row

Add a new column:

- Add a column Gender with values taken from the user.

- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.

- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.

- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")

```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = ['Age'])\
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Output:

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Average time: 0.065 s
Maximum time: 0.065 s
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1
Expected output
Original DataFrame:
..... Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
New name: Susan
New age: 29
After adding a row:
..... Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
3 Susan 29
Index of row to modify: 2
New age: 27
After modifying a row:
..... Name Age
0 Alice 27
1 Bob 30
2 Charlie 35
3 Susan 29
Index of row to delete: 1
Actual output
Original DataFrame:
..... Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
New name: Susan
New age: 29
After adding a row:
..... Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
3 Susan 29
Index of row to modify: 2
New age: 27
After modifying a row:
..... Name Age
0 Alice 27
1 Bob 30
2 Charlie 35
3 Susan 29
Index of row to delete: 1

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

Display the first five rows of the data frame.

Calculate the average age of the students (limit the average age up to 2 decimal places).

Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# write your code here..
```

```
print("First five rows:")
```

```
print(data.head())
```

```
average_age = round(data["Age"].mean(), 2)
```

```
print(f"Average age: {average_age}")
```

```
filtered_students = data[data["Grade"] <= "B"]
```

```
print("Students with a grade up to B")
```

```
print(filtered_students)
```

Output:

The screenshot displays a coding interface with three main sections:

- Problem Statement:** Titled "4.1.3. Student Information", it describes the task of reading a text file and performing three operations: displaying the first five rows, calculating the average age (rounded to 2 decimal places), and filtering students by grade (up to 'B').
- Code Editor:** Contains the Python code using Pandas to read a CSV file, calculate the average age, and filter students based on their grade.
- Test Results:** Shows the execution of the code against test cases. It indicates that 1 out of 1 shown test case(s) passed and 1 out of 1 hidden test case(s) passed. The output for the first test case is displayed, showing the first five rows of the data frame, the average age (23.29), and the filtered list of students with a grade up to 'B'.

Expected output:

```
First five rows:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
3  Mike  23    C
4  Sony  21    B
Average age: 23.29
Students with a grade up to B:
  Name Age Grade
0  John  25    A
1  Allen  22    B
2  Emma  24    A
4  Sony  21    B
5  Aelia  26    A
```

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

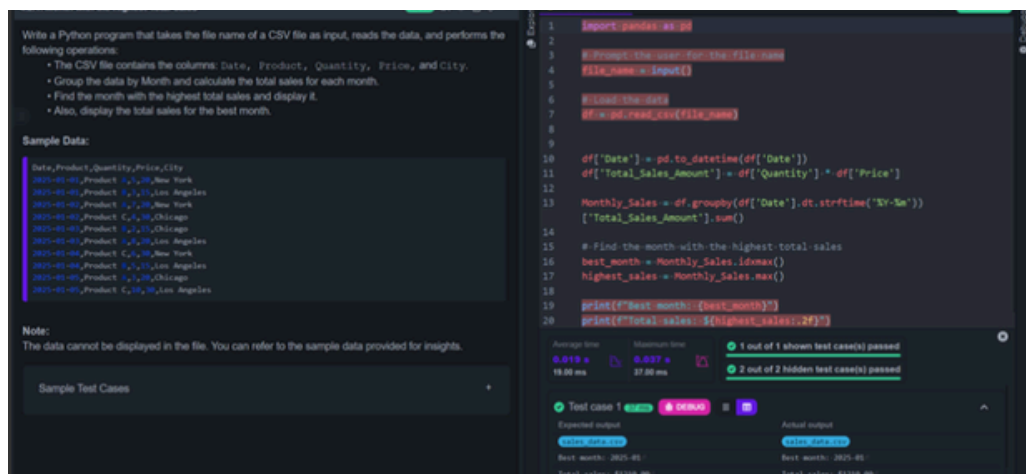
- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:



The screenshot shows a code editor with a Python program on the left and its output on the right. The program prompts the user for a file name, reads the data, and calculates the total sales for each month. The output shows the best month and total sales.

Code:

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 df['Date'] = pd.to_datetime(df['Date'])
11 df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
12
13 Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))
14   ['Total_Sales_Amount'].sum()
15
16 # Find the month with the highest total sales
17 best_month = Monthly_Sales.idxmax()
18 highest_sales = Monthly_Sales.max()
19
20 print(f"Best month: {best_month}")
21 print(f"Total sales: ${highest_sales:.2f}")
```

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	P_1	10	10	New York
2023-01-01	P_2	5	20	Los Angeles
2023-01-01	P_3	15	10	New York
2023-01-01	P_4	10	10	Chicago
2023-01-01	P_5	10	10	Los Angeles
2023-01-01	P_6	10	10	New York
2023-01-01	P_7	10	10	Los Angeles
2023-01-01	P_8	10	10	Chicago
2023-01-01	P_9	10	10	Los Angeles
2023-01-01	P_10	10	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case 1: **Pass**

Test Case 2: **Pass**

Test Case 3: **Pass**

Test Case 4: **Pass**

Test Case 5: **Pass**

Test Case 6: **Pass**

Test Case 7: **Pass**

Test Case 8: **Pass**

Test Case 9: **Pass**

Test Case 10: **Pass**

Test Case 11: **Pass**

Test Case 12: **Pass**

Test Case 13: **Pass**

Test Case 14: **Pass**

Test Case 15: **Pass**

Test Case 16: **Pass**

Test Case 17: **Pass**

Test Case 18: **Pass**

Test Case 19: **Pass**

Test Case 20: **Pass**

Test Case 21: **Pass**

Test Case 22: **Pass**

Test Case 23: **Pass**

Test Case 24: **Pass**

Test Case 25: **Pass**

Test Case 26: **Pass**

Test Case 27: **Pass**

Test Case 28: **Pass**

Test Case 29: **Pass**

Test Case 30: **Pass**

Test Case 31: **Pass**

Test Case 32: **Pass**

Test Case 33: **Pass**

Test Case 34: **Pass**

Test Case 35: **Pass**

Test Case 36: **Pass**

Test Case 37: **Pass**

Test Case 38: **Pass**

Test Case 39: **Pass**

Test Case 40: **Pass**

Test Case 41: **Pass**

Test Case 42: **Pass**

Test Case 43: **Pass**

Test Case 44: **Pass**

Test Case 45: **Pass**

Test Case 46: **Pass**

Test Case 47: **Pass**

Test Case 48: **Pass**

Test Case 49: **Pass**

Test Case 50: **Pass**

Test Case 51: **Pass**

Test Case 52: **Pass**

Test Case 53: **Pass**

Test Case 54: **Pass**

Test Case 55: **Pass**

Test Case 56: **Pass**

Test Case 57: **Pass**

Test Case 58: **Pass**

Test Case 59: **Pass**

Test Case 60: **Pass**

Test Case 61: **Pass**

Test Case 62: **Pass**

Test Case 63: **Pass**

Test Case 64: **Pass**

Test Case 65: **Pass**

Test Case 66: **Pass**

Test Case 67: **Pass**

Test Case 68: **Pass**

Test Case 69: **Pass**

Test Case 70: **Pass**

Test Case 71: **Pass**

Test Case 72: **Pass**

Test Case 73: **Pass**

Test Case 74: **Pass**

Test Case 75: **Pass**

Test Case 76: **Pass**

Test Case 77: **Pass**

Test Case 78: **Pass**

Test Case 79: **Pass**

Test Case 80: **Pass**

Test Case 81: **Pass**

Test Case 82: **Pass**

Test Case 83: **Pass**

Test Case 84: **Pass**

Test Case 85: **Pass**

Test Case 86: **Pass**

Test Case 87: **Pass**

Test Case 88: **Pass**

Test Case 89: **Pass**

Test Case 90: **Pass**

Test Case 91: **Pass**

Test Case 92: **Pass**

Test Case 93: **Pass**

Test Case 94: **Pass**

Test Case 95: **Pass**

Test Case 96: **Pass**

Test Case 97: **Pass**

Test Case 98: **Pass**

Test Case 99: **Pass**

Test Case 100: **Pass**

Test Case 101: **Pass**

Test Case 102: **Pass**

Test Case 103: **Pass**

Test Case 104: **Pass**

Test Case 105: **Pass**

Test Case 106: **Pass**

Test Case 107: **Pass**

Test Case 108: **Pass**

Test Case 109: **Pass**

Test Case 110: **Pass**

Test Case 111: **Pass**

Test Case 112: **Pass**

Test Case 113: **Pass**

Test Case 114: **Pass**

Test Case 115: **Pass**

Test Case 116: **Pass**

Test Case 117: **Pass**

Test Case 118: **Pass**

Test Case 119: **Pass**

Test Case 120: **Pass**

Test Case 121: **Pass**

Test Case 122: **Pass**

Test Case 123: **Pass**

Test Case 124: **Pass**

Test Case 125: **Pass**

Test Case 126: **Pass**

Test Case 127: **Pass**

Test Case 128: **Pass**

Test Case 129: **Pass**

Test Case 130: **Pass**

Test Case 131: **Pass**

Test Case 132: **Pass**

Test Case 133: **Pass**

Test Case 134: **Pass**

Test Case 135: **Pass**

Test Case 136: **Pass**

Test Case 137: **Pass**

Test Case 138: **Pass**

Test Case 139: **Pass**

Test Case 140: **Pass**

Test Case 141: **Pass**

Test Case 142: **Pass**

Test Case 143: **Pass**

Test Case 144: **Pass**

Test Case 145: **Pass**

Test Case 146: **Pass**

Test Case 147: **Pass**

Test Case 148: **Pass**

Test Case 149: **Pass**

Test Case 150: **Pass**

Test Case 151: **Pass**

Test Case 152: **Pass**

Test Case 153: **Pass**

Test Case 154: **Pass**

Test Case 155: **Pass**

Test Case 156: **Pass**

Test Case 157: **Pass**

Test Case 158: **Pass**

Test Case 159: **Pass**

Test Case 160: **Pass**

Test Case 161: **Pass**

Test Case 162: **Pass**

Test Case 163: **Pass**

Test Case 164: **Pass**

Test Case 165: **Pass**

Test Case 166: **Pass**

Test Case 167: **Pass**

Test Case 168: **Pass**

Test Case 169: **Pass**

Test Case 170: **Pass**

Test Case 171: **Pass**

Test Case 172: **Pass**

Test Case 173: **Pass**

Test Case 174: **Pass**

Test Case 175: **Pass**

Test Case 176: **Pass**

Test Case 177: **Pass**

Test Case 178: **Pass**

Test Case 179: **Pass**

Test Case 180: **Pass**

Test Case 181: **Pass**

Test Case 182: **Pass**

Test Case 183: **Pass**

Test Case 184: **Pass**

Test Case 185: **Pass**

Test Case 186: **Pass**

Test Case 187: **Pass**

Test Case 188: **Pass**

Test Case 189: **Pass**

Test Case 190: **Pass**

Test Case 191: **Pass**

Test Case 192: **Pass**

Test Case 193: **Pass**

Test Case 194: **Pass**

Test Case 195: **Pass**

Test Case 196: **Pass**

Test Case 197: **Pass**

Test Case 198: **Pass**

Test Case 199: **Pass**

Test Case 200: **Pass**

Test Case 201: **Pass**

Test Case 202: **Pass**

Test Case 203: **Pass**

Test Case 204: **Pass**

Test Case 205: **Pass**

Test Case 206: **Pass**

Test Case 207: **Pass**

Test Case 208: **Pass**

Test Case 209: **Pass**

Test Case 210: **Pass**

Test Case 211: **Pass**

Test Case 212: **Pass**

Test Case 213: **Pass**

Test Case 214: **Pass**

Test Case 215: **Pass**

Test Case 216: **Pass**

Test Case 217: **Pass**

Test Case 218: **Pass**

Test Case 219: **Pass**

Test Case 220: **Pass**

Test Case 221: **Pass**

Test Case 222: **Pass**

Test Case 223: **Pass**

Test Case 224: **Pass**

Test Case 225: **Pass**

Test Case 226: **Pass**

Test Case 227: **Pass**

Test Case 228: **Pass**

Test Case 229: **Pass**

Test Case 230: **Pass**

Test Case 231: **Pass**

Test Case 232: **Pass**

Test Case 233: **Pass**

Test Case 234: **Pass**

Test Case 235: **Pass**

Test Case 236: **Pass**

Test Case 237: **Pass**

Test Case 238: **Pass**

Test Case 239: **Pass**

Test Case 240: **Pass**

Test Case 241: **Pass**

Test Case 242: **Pass**

Test Case 243: **Pass**

Test Case 244: **Pass**

Test Case 245: **Pass**

Test Case 246: **Pass**

Test Case 247: **Pass**

Test Case 248: **Pass**

Test Case 249: **Pass**

Test Case 250: **Pass**

Test Case 251: **Pass**

Test Case 252: **Pass**

Test Case 253: **Pass**

Test Case 254: **Pass**

Test Case 255: **Pass**

Test Case 256: **Pass**

Test Case 257: **Pass**

Test Case 258: **Pass**

Test Case 259: **Pass**

Test Case 260: **Pass**

Test Case 261: **Pass**

Test Case 262: **Pass**

Test Case 263: **Pass**

Test Case 264: **Pass**

Test Case 265: **Pass**

Test Case 266: **Pass**

Test Case 267: **Pass**

Test Case 268: **Pass**

Test Case 269: **Pass**

Test Case 270: **Pass**

Test Case 271: **Pass**

Test Case 272: **Pass**

Test Case 273: **Pass**

Test Case 274: **Pass**

Test Case 275: **Pass**

Test Case 276: **Pass**

Test Case 277: **Pass**

Test Case 278: **Pass**

Test Case 279: **Pass**

Test Case 280: **Pass**

Test Case 281: **Pass**

Test Case 282: **Pass**

Test Case 283: **Pass**

Test Case 284: **Pass**

Test Case 285: **Pass**

Test Case 286: **Pass**

Test Case 287: **Pass**

Test Case 288: **Pass**

Test Case 289: **Pass**

Test Case 290: **Pass**

Test Case 291: **Pass**

Test Case 292: **Pass**

Test Case 293: **Pass**

Test Case 294: **Pass**

Test Case 295: **Pass**

Test Case 296: **Pass**

Test Case 297: **Pass**

Test Case 298: **Pass**

Test Case 299: **Pass**

Test Case 300: **Pass**

Test Case 301: **Pass**

Test Case 302: **Pass**

Test Case 303: **Pass**

Test Case 304: **Pass**

Test Case 305: **Pass**

Test Case 306: **Pass**

Test Case 307: **Pass**

Test Case 308: **Pass**

Test Case 309: **Pass**

Test Case 310: **Pass**

Test Case 311: **Pass**

Test Case 312: **Pass**

Test Case 313: **Pass**

Test Case 314: **Pass**

Test Case 315: **Pass**

Test Case 316: **Pass**

Test Case 317: **Pass**

Test Case 318: **Pass**

Test Case 319: **Pass**

Test Case 320: **Pass**

Test Case 321: **Pass**

Test Case 322: **Pass**

Test Case 323: **Pass**

Test Case 324: **Pass**

Test Case 325: **Pass**

Test Case 326: **Pass**

Test Case 327: **Pass**

Test Case 328: **Pass**

Test Case 329: **Pass**

Test Case 330: **Pass**

Test Case 331: **Pass**

Test Case 332: **Pass**

Test Case 333: **Pass**

Test Case 334: **Pass**

Test Case 335: **Pass**

Test Case 336: **Pass**

Test Case 337: **Pass**

Test Case 338: **Pass**

Test Case 339: **Pass**

Test Case 340: **Pass**

Test Case 341: **Pass**

Test Case 342: **Pass**

Test Case 343: **Pass**

Test Case 344: **Pass**

Test Case 345: **Pass**

Test Case 346: **Pass**

Test Case 347: **Pass**

Test Case 348: **Pass**

Test Case 349: **Pass**

Test Case 350: **Pass**

Test Case 351: **Pass**

Test Case 352: **Pass**

Test Case 353: **Pass**

Test Case 354: **Pass**

Test Case 355: **Pass**

Test Case 356: **Pass**

Test Case 357: **Pass**

Test Case 358: **Pass**

Test Case 359: **Pass**

Test Case 360: **Pass**

Test Case 361: **Pass**

Test Case 362: **Pass**

Test Case 363: **Pass**

Test Case 364: **Pass**

Test Case 365: **Pass**

Test Case 366: **Pass**

Test Case 367: **Pass**

Test Case 368: **Pass**

Test Case 369: **Pass**

Test Case 370: **Pass**

Test Case 371: **Pass**

Test Case 372: **Pass**

Test Case 373: **Pass**

Test Case 374: **Pass**

Test Case 375: **Pass**

Test Case 376: **Pass**

Test Case 377: **Pass**

Test Case 378: **Pass**

Test Case 379: **Pass**

Test Case 380: **Pass**

Test Case 381: **Pass**

Test Case 382: **Pass**

Test Case 383: **Pass**

Test Case 384: **Pass**

Test Case 385: **Pass**

Test Case 386: **Pass**

Test Case 387: **Pass**

Test Case 388: **Pass**

Test Case 389: **Pass**

Test Case 390: **Pass**

Test Case 391: **Pass**

Test Case 392: **Pass**

Test Case 393: **Pass**

Test Case 394: **Pass**

Test Case 395: **Pass**

Test Case 396: **Pass**

Test Case 397: **Pass**

Test Case 398: **Pass**

Test Case 399: **Pass**

Test Case 400: **Pass**

Test Case 401: **Pass**

Test Case 402: **Pass**

Test Case 403: **Pass**

Test Case 404: **Pass**

Test Case 405: **Pass**

Test Case 406: **Pass**

Test Case 407: **Pass**

Test Case 408: **Pass**

Test Case 409: **Pass**

Test Case 410: **Pass**

Test Case 411: **Pass**

Test Case 412: **Pass**

Test Case 413: **Pass**

Test Case 414: **Pass**

Test Case 415: **Pass**

Test Case 416: **Pass**

Test Case 417: **Pass**

Test Case 418: **Pass**

Test Case 419: **Pass**

Test Case 420: **Pass**

Test Case 421: **Pass**

Test Case 422: **Pass**

Test Case 423: **Pass**

Test Case 424: **Pass**

Test Case 425: **Pass**

Test Case 426: **Pass**

Test Case 427: **Pass**

Test Case 428: **Pass**

Test Case 429: **Pass**

Test Case 430: **Pass**

Test Case 431: **Pass**

Test Case 432: **Pass**

Test Case 433: **Pass**

Test Case 434: **Pass**

Test Case 435: **Pass**

Test Case 436: **Pass**

Test Case 437: **Pass**

Test Case 438: **Pass**

Test Case 439: **Pass**

Test Case 440: **Pass**

Test Case 441: **Pass**

Test Case 442: **Pass**

Test Case 443: **Pass**

Test Case 444: **Pass**

Test Case 445: **Pass**

Test Case 446: **Pass**

Test Case 447: **Pass**

Test Case 448: **Pass**

Test Case 449: **Pass**

Test Case 450: **Pass**

Test Case 451: **Pass**

Test Case 452: **Pass**

Test Case 453: **Pass**

Test Case 454: **Pass**

Test Case 455: **Pass**

Test Case 456: **Pass**

Test Case 457: **Pass**

Test Case 458: **Pass**

Test Case 459: **Pass**

Test Case 460: **Pass**

Test Case 461: **Pass**

Test Case 462: **Pass**

Test Case 463: **Pass**

Test Case 464: **Pass**

Test Case 465: **Pass**

Test Case 466: **Pass**

Test Case 467: **Pass**

Test Case 468: **Pass**

Test Case 469: **Pass**

Test Case 470: **Pass**

Test Case 471: **Pass**

Test Case 472: **Pass**

Test Case 473: **Pass**

Test Case 474: **Pass**

Test Case 475: **Pass**

Test Case 476: **Pass**

Test Case 477: **Pass**

Test Case 478: **Pass**

Test Case 479: **Pass**

Test Case 480: **Pass**

Test Case 481: **Pass**

Test Case 482: **Pass**

Test Case 483: **Pass**

Test Case 484: **Pass**

Test Case 485: **Pass**

Test Case 486: **Pass**

Test Case 487: **Pass**

Test Case 488: **Pass**

Test Case 489: **Pass**

Test Case 490: **Pass**

Test Case 491: **Pass**

Test Case 492: **Pass**

Test Case 493: **Pass**

Test Case 494: **Pass**

Test Case 495: **Pass**

Test Case 496: **Pass**

Test Case 497: **Pass**

Test Case 498: **Pass**

Test Case 499: **Pass**

Test Case 500: **Pass**

Test Case 501: **Pass**

Test Case 502: **Pass**

Test Case 503: **Pass**

Test Case 504: **Pass**

Test Case 505: **Pass</**

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Find the product that sold the most in terms of quantity sold.
Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
product_sales = df.groupby("Product")["Quantity"].sum()
```

```
# Find the product with the highest total quantity sold
```

```
best_product = product_sales.idxmax()
```

```
highest_quantity = product_sales.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```

Output:

The screenshot displays a code editor with a Python script and its execution results. The script prompts for a file name, loads a CSV file, groups data by product to find the highest quantity sold, and prints the results. The output shows 'Product A' as the best selling product with a total quantity of 25.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	10	20	New York
2023-01-01	Product A	5	15	Los Angeles
2023-01-01	Product A	10	20	New York
2023-01-01	Product C	5	10	Chicago
2023-01-01	Product B	5	10	Chicago
2023-01-01	Product A	5	10	Los Angeles
2023-01-01	Product C	5	10	New York
2023-01-01	Product A	5	10	Los Angeles
2023-01-01	Product A	5	10	Chicago
2023-01-01	Product C	5	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case	Expected Output	Actual Output
Test case 1	Best selling product: Product A Total quantity sold: 25	Best selling product: Product A Total quantity sold: 25

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# write the code..
```

```
city_sales = df.groupby("City")["Quantity"].sum()
```

```
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

Output:

The screenshot displays a code editor with the following content:

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	10	\$10	New York
2023-01-01	Product A	20	\$20	Los Angeles
2023-01-01	Product A	30	\$30	New York
2023-01-01	Product C	10	\$10	Chicago
2023-01-01	Product A	20	\$20	Chicago
2023-01-01	Product A	20	\$20	Los Angeles
2023-01-01	Product C	10	\$10	New York
2023-01-01	Product A	20	\$20	Los Angeles
2023-01-01	Product A	10	\$10	Chicago
2023-01-01	Product C	10	\$10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case 1: Expected output: City sold the most products: Los Angeles. Actual output: City sold the most products: Los Angeles.

Test Case 2: Expected output: City sold the most products: Los Angeles. Actual output: City sold the most products: Los Angeles.

Test Results: 1 out of 1 shown test case(s) passed. 2 out of 2 hidden test case(s) passed.

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd

from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
grouped = df.groupby("Date")["Product"].apply(list)

pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)

max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==
max_frequency]

# Output the most frequent product pairs
for (product1, product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

Output:

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	1	10	New York
2025-01-01	Product B	2	15	Los Angeles
2025-01-02	Product A	2	20	New York
2025-01-02	Product C	1	10	Chicago
2025-01-03	Product A	2	10	Chicago
2025-01-03	Product B	1	10	Los Angeles
2025-01-04	Product C	1	10	New York
2025-01-04	Product A	1	10	Los Angeles
2025-01-05	Product C	2	20	Chicago
2025-01-05	Product B	1	10	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)

Frequency

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 grouped = df.groupby("Date")["Product"].apply(list)
13
14 pairs_counter = Counter()
15 for products in grouped:
16     pairs = combinations(sorted(products), 2)
17     pairs_counter.update(pairs)
18
```

Average time

0.015 s

Maximum time

0.025 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Product A and Product B: 2 times
Product A and Product C: 2 times

Actual output

Product A and Product B: 2 times
Product A and Product C: 2 times

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())
```

```
# 1. Display the first 5 rows of the dataset
print(data.tail())
```

```
# 2. Display the last 5 rows of the dataset
print(data.shape)
```

```
# 3. Get the shape of the dataset
print(data.info())
```

```
# 4. Get a summary of the dataset (info)
print(data.describe())
```

```
# 5. Get basic statistics of the dataset
print(data.isnull().sum())
```

```
# 6. Check for missing value
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace = True)
```

```
# 7. Fill missing values in the 'Age' column with the median age
mode_embarked = data['Embarked'].mode() [0]
data['Embarked'].fillna(mode_embarked, inplace = True)
```

```
# 8. Fill missing values in the 'Embarked' column with the mode
data.drop('Cabin', axis=1, inplace = True)
```

```
# 9. Drop the 'Cabin' column due to many missing values
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

Output:

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown	male	21	1	0	A/5 21171	5.29		S
2	1	1	Mr. W. H. Bradley	male	30	0	0	PC 17599	71.2833	C85	C
3	1	3	Mr. O'Sullivan	male	34	0	0	Q 3309	16.9958		S
4	0	3	Mr. O'Sullivan	male	34	0	0	Q 3309	16.9958		S
5	1	1	Mr. W. H. Bradley	male	30	0	0	PC 17599	71.2833	C85	C

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, male, 21, 1, 0, A/5 21171, 5.29, , S
2, 1, 1, Mr. W. H. Bradley, male, 30, 0, 0, PC 17599, 71.2833, C85, C
3, 1, 3, Mr. O'Sullivan, male, 34, 0, 0, Q 3309, 16.9958, , S
4, 0, 3, Mr. O'Sullivan, male, 34, 0, 0, Q 3309, 16.9958, , S
5, 1, 1, Mr. W. H. Bradley, male, 30, 0, 0, PC 17599, 71.2833, C85, C
```

Execution Summary: 1 out of 1 shown test case(s) passed

Test case 1: Expected output vs Actual output comparison.

Expected output (5 rows x 12 columns):

```
PassengerId Survived Pclass Fare Cabin Embarked
0 0 1 3 7.2500 S
1 1 2 1 71.2833 C85
2 1 2 1 7.5250 S
3 1 4 1 53.5800 C123
4 1 5 1 8.4500 S
```

Actual output (5 rows x 12 columns):

```
PassengerId Survived Pclass Fare Cabin Embarked
0 0 1 3 7.2500 S
1 1 2 1 71.2833 C85
2 1 2 1 7.5250 S
3 1 4 1 53.5800 C123
4 1 5 1 8.4500 S
```

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)
```

```
# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)
```

```
# 5. Get the median fare of passengers
```

```
median_fare = data['Fare'].median()
print(median_fare)
```

```
# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

```
# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

```
# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 9. Calculate the survival rate
survival_rate = data['Survived'].mean()
print(survival_rate)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

The screenshot displays a Jupyter Notebook environment with a dark theme. On the left, the notebook contains a title '4.2.6. Titanic Dataset Analysis and Data Cleaning - 2', a description of the task, a list of 10 questions, a table of the dataset's columns, and a sample of the data. The code in the notebook cells matches the provided code blocks. On the right, the 'Interactive Console' shows the execution of the code, with a green status bar indicating '1 out of 1 shown test case(s) passed'. Below this, a comparison between 'Expected output' and 'Actual output' is shown for several variables, including Pclass, Sex, and Survived, all of which match perfectly. The bottom of the console shows a 'Test cases' button and a 'Terminated' status.

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. Brown	male	22	1	0	21171, 7, 21, 5	5		S
2	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
3	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
4	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
5	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
6	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
7	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
8	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
9	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S
10	1	1	Mr. Brown, Mr. Brown	female	18	0	0	PC 17599, 71, 2811, CB, C	5		S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mr. Brown, male, 22, 1, 0, 21171, 7, 21, S
2, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
3, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
4, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
5, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
6, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
7, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
8, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
9, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
10, 1, 1, Mr. Brown, Mr. Brown, female, 18, 0, 0, PC 17599, 71, 2811, CB, C
```

Interactive Console

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
```

Average time: 0.000 s, Maximum time: 0.000 s, 1 out of 1 shown test case(s) passed

Test case 1: Expected output vs Actual output

Expected output	Actual output
29.69911764795882	29.69911764795882
14.4542	14.4542
1 - 491	1 - 491
1 - 216	1 - 216
2 - 184	2 - 184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0 - 577	0 - 577
1 - 114	1 - 114
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0 - 549	0 - 549
1 - 342	1 - 342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0 - 3838181818181818	0 - 3838181818181818
Sex	Sex
0 - 0.189808	0 - 0.189808
1 - 0.742018	1 - 0.742018
Name: Survived, dtype: float64	Name: Survived, dtype: float64

Test cases

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
```

```
print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

The screenshot displays a Jupyter Notebook titled "4.2.7. Titanic Dataset Analysis and Data Cleaning - 3". The notebook contains a list of 10 tasks for analysis and a corresponding output window showing the results of the first test case.

Tasks:

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
2	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
3	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
4	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
5	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
6	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
7	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
8	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
9	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S
10	1	1	Mr. Brown, Mr. William Henry	male	22	1	0	A/5 21171/2	5.29	5	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, ParCh, Ticket, Fare, Cabin, Embarked
```

Test case 1 output:

Expected output	Actual output
Pclass:	Pclass:
1 - 0.629630	1 - 0.629630
2 - 0.472826	2 - 0.472826
3 - 0.242363	3 - 0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S:	Embarked_S:
0 - 0.586873	0 - 0.586873
1 - 0.338957	1 - 0.338957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize:	FamilySize:
0 - 0.383538	0 - 0.383538
1 - 0.552795	1 - 0.552795
2 - 0.578411	2 - 0.578411
3 - 0.724138	3 - 0.724138
4 - 0.200000	4 - 0.200000
5 - 0.136364	5 - 0.136364
6 - 0.333333	6 - 0.333333
7 - 0.400000	7 - 0.400000
8 - 0.400000	8 - 0.400000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone:	IsAlone:
0 - 0.585658	0 - 0.585658
1 - 0.383538	1 - 0.383538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass:	Pclass:

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
```

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
```

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
```

```
non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

```
# 5. Calculate the percentage of children (Age < 18) who survived
```

```
children = data[data['Age'] < 18]
```

```
children_survival_rate = children['Survived'].mean()
```

```
print(children_survival_rate)
```

```
# 6. Calculate the percentage of adults (Age >= 18) who survived
```

```
adults = data[data['Age'] >= 18]
```

```
adults_survival_rate = adults['Survived'].mean()
```

```
print(adults_survival_rate)
```

```
# 7. Get the median age of survivors
```

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
```

```
print(median_age_survivors)
```

```
# 8. Get the median age of non-survivors
```

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
```

```
print(median_age_non_survivors)
```

```
# 9. Get the median fare of survivors
```

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
```

```
print(median_fare_survivors)
```

```
# 10. Get the median fare of non-survivors
```

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
```

```
print(median_fare_non_survivors)
```

Output:

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, S
2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs) Bower", female, 38, 1, 0, PC 17599, 53.1, C85, C
3, 1, 3, "Heikkinen, Mrs. (Elis) V. S. (Mrs.)", female, 26, 0, 0, STON/O 3101282, 53.1, S
4, 1, 1, "Allouart, Mrs. (Julia) (Mrs.)", female, 19, 1, 0, 113803, 53.1, C123, S
5, 0, 3, "Smyth, Mr. (Paddy) (Mrs.)", male, 28, 0, 0, 330957, 51.66, S
6, 0, 3, "Palsson, Mr. (Gustaf) (Mrs.)", male, 34, 0, 0, 17463, 51.66, S
```

TitanicDat...

1 import pandas as pd

2 import numpy as np

3

4 # Load the Titanic dataset

Average time: 0.049 s, Maximum time: 49.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Female: 233

Female: 233

Male: 309

Male: 309

Name: Sex, dtype: int64

Name: Sex, dtype: int64

Male: 408

Male: 408

Female: 81

Female: 81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

1: 217

1: 217

0: 125

0: 125

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

1: 427

1: 427

0: 122

0: 122

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

0.5398238888495575

0.5398238888495575

0.381815119767905

0.381815119767905

28.0

28.0

28.0

28.0

28.0

28.0